ThaparTrade

Prototype Stage Report for UCS503P
Submitted to: Dr. Asif
Thapar Institute of Engineering and Technology

Rajbir Singh, CSED* Chitesh Jindal, CSED† Ishan Garg, CSED‡

September 14, 2026

1 Higher-order goal

This project provides Thapar students with a centralized, structured and campus-focused marketplace for buying, selling and exchanging products within the student community. The prototype connects authenticated users, product listing management, marketplace discovery, search and filtering, listing status management, reporting, and administrative moderation into a single web platform.

The wider goal is to replace fragmented informal channels such as WhatsApp groups, social media, personal contacts and scattered messages with a dedicated marketplace that improves discoverability, organization and trust.

2 Time-to-value

- We delivered the core *register, list, discover, manage and report* workflow before considering advanced marketplace functionality.
- Development was organized around the major marketplace modules: authentication, listing CRUD, browsing, search and filtering, listing status, profiles, reporting and moderation.
- Success in this iteration is a working prototype where a student can authenticate, create a product listing, make it discoverable in the marketplace, manage its status, and report problematic listings through the platform.

3 Problem Statement

Student-to-student buying and selling at Thapar is commonly conducted through informal channels such as WhatsApp groups, social media, personal contacts and word of mouth. These channels are convenient but do not provide a structured marketplace workflow.

Common pain points include:

- **Scattered listings:** products are distributed across multiple groups, chats and personal networks.
- **Poor discoverability:** useful listings can disappear quickly among unrelated messages, making relevant products difficult to locate.
- **Limited search and filtering:** buyers cannot consistently search products by category, price, condition or availability.
- **Stale availability:** sellers may not update or remove listings after an item has been sold.
- **Trust concerns:** buyers need confidence in the seller and the information provided about a product.
- **Lack of moderation:** there is no centralized campus-specific mechanism for reporting and reviewing misleading, unauthorized or problematic listings.

Why this matters now: students need a single, searchable and organized platform through which they can discover products, publish listings, communicate with other students and maintain accurate availability information.

4 Proposed Solution

4.1 Overview

A web-based campus marketplace where verified Thapar users can create, discover and manage product listings. The system provides a marketplace interface, structured listing information, search and filtering, listing status management, user profiles, reporting and administrative moderation.

The platform is designed around a simple student-to-student trading workflow while keeping the core application modular enough for future extensions.

4.2 Core workflow

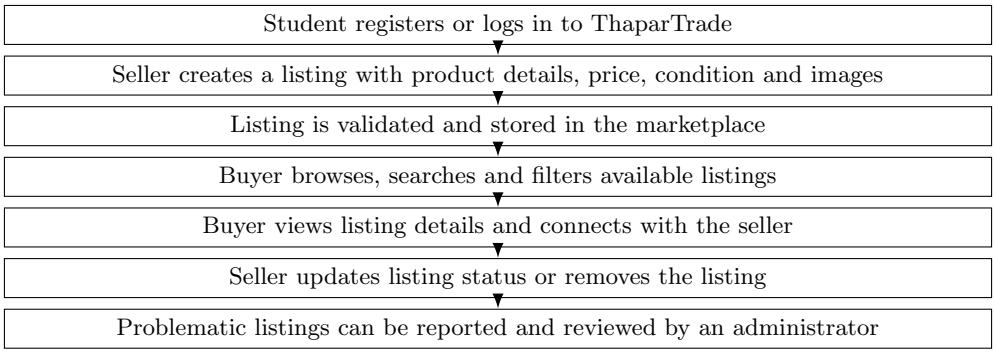


Figure 1: End-to-end workflow of the ThaparTrade prototype.

4.3 Current prototype and implementation status

The current ThaparTrade prototype provides the core campus marketplace functionality. Users can register and log in, manage their profiles, browse listings, search and filter products, view listing details, create listings, edit and delete their own listings, and update listing status.

The administrative workflow supports viewing reports, reviewing reported listings, and removing or restoring listings.

The project scope covers user authentication, listing creation and management, marketplace browsing, search and filtering, listing status management, user profile management, reporting and moderation, and persistent data storage.

5 Solution Approach

This is an engineering course project, so the emphasis is on a working, testable campus marketplace rather than novel algorithm research. The solution is divided into independently understandable functional modules.

5.1 Marketplace and listing engine

The listing workflow is structured around product information and availability.

A seller provides the product title, category, description, price, condition and images. The system stores the listing and exposes it through the marketplace. The seller can subsequently edit, delete or update the status of the listing.

The principal listing states are:

AVAILABLE $\rightarrow$ **RESERVED** $\rightarrow$ **SOLD**

This explicit lifecycle reduces stale availability and provides buyers with clearer information about the current state of an item.

5.2 Web architecture

A 3-tier web architecture is used:

- **Frontend:** React.js and Tailwind CSS provide the responsive marketplace, listing interfaces, search and filtering, user profile functionality and administrative views.
- **Backend API:** Node.js and Express.js provide the application services and REST-style communication between the frontend and the data layer.
- **Database:** MongoDB provides persistent storage for users, listings and marketplace records.

The documented technology stack additionally includes JWT for authentication, AWS S3 for image storage and Razorpay for payment-related marketplace functionality.

5.3 Listing and moderation status

The marketplace separates normal student operations from administrative moderation.

A student can report a listing that appears misleading, unauthorized or otherwise problematic. The report is stored and made available to the administrator for review.

The administrator can inspect the report and take an appropriate action, including removing or restoring the listing.

6 Evaluation Criterion

6.1 Primary evaluation metric

Listing-to-discovery efficiency (LDE): time required for a user to locate a specified relevant product using the marketplace search and filtering functionality.

- **Target:** the majority of pilot users should be able to locate a specified listing within a short predefined task window.
- **Attribution:** task completion time can be recorded during user-acceptance testing.

6.2 Secondary evaluation metrics

- **Listing completion rate:** percentage of users who successfully publish a valid listing.
- **Search success rate:** percentage of users who successfully find the intended item during predefined search tasks.
- **Usability:** average user rating of navigation, marketplace browsing and listing creation on a 1–5 scale.
- **Listing freshness:** percentage of sampled listings whose Available, Reserved or Sold state correctly reflects their current status.
- **Reliability:** successful execution of authentication, listing creation, search, update and reporting operations.
- **Performance:** response time of common marketplace and listing operations under expected student workload.

6.3 Pilot validation plan

- Recruit a small group of student users representing both buyers and sellers.
- Provide predefined tasks such as finding a product below a specified price or creating a listing for a used item.
- Record task completion, errors, time taken and user feedback.
- Use the results to refine the marketplace interface and workflow before the final demonstration.

7 Scalability

1. **Scalable architecture:** the frontend, backend and database responsibilities are separated, allowing individual components to evolve independently as the number of users and listings increases.
2. **Marketplace growth:** database indexing, pagination and efficient search queries can support increasing numbers of listings.
3. **Image scalability:** product images can be stored separately from core transactional data using cloud object storage such as AWS S3.
4. **Modular deployment:** the application can be deployed on common web-hosting infrastructure and migrated to larger infrastructure as usage grows.

8 Engine availability heuristic

The prototype uses mature and commonly available web-development technologies rather than requiring novel infrastructure.

React.js and Tailwind CSS provide the frontend, while Node.js and Express.js provide the backend API. MongoDB provides persistent application data storage. JWT provides authenticated access, while GitHub is used for version control and team collaboration.

The documented technology stack also includes AWS S3 for image storage and Razorpay for payment-related functionality.

This technology combination allows the team to focus on marketplace workflow, usability, listing management and moderation rather than building infrastructure from scratch.

9 Project scope and deliverables

9.1 Initial deliverable status

- **User registration and login:** core authentication functionality.
- **Marketplace interface:** browsing of available product listings.
- **Listing creation:** structured product listing with description, price, condition and images.
- **Listing management:** edit and delete functionality for user-owned listings.
- **Search and filtering:** keyword-based discovery with marketplace filtering.
- **Listing status:** Available, Reserved and Sold lifecycle management.
- **Profile management:** user profile functionality and personal listing management.
- **Reporting and moderation:** report submission and administrative listing review.

9.2 Subsequent deliverables

- Improved search, filtering and sorting behaviour.
- Enhanced administrative moderation dashboard and report records.
- Improved image handling and listing previews.
- Optional in-platform messaging or structured buyer-seller communication.
- Notifications for listing status and relevant user actions.
- Improved analytics for marketplace usage and listing activity.
- Further testing with student users and refinement of the integrated prototype.

10 Risks and mitigations

- **Misleading listings:** provide reporting reasons and an administrative review workflow so problematic listings can be investigated.
- **Stale availability:** use explicit Available, Reserved and Sold states and allow sellers to update listing status.
- **Incomplete listings:** validate required product information before allowing a listing to be published.
- **Trust concerns:** restrict marketplace functionality to authenticated users and provide reporting and moderation mechanisms.
- **Integration defects:** perform feature-level tests followed by integrated testing across the frontend, backend and database.
- **Limited development resources:** prioritize the complete core marketplace before advanced functionality such as notifications, analytics and additional communication features.

11 Summary

The ThaparTrade prototype delivers a centralized campus marketplace for student-to-student buying and selling. It provides authentication, listing creation and management, marketplace browsing, search and filtering, listing status management, user profiles, reporting and administrative moderation.

The project addresses the fragmentation of informal student trading channels by providing a structured workflow for publishing and discovering products. The separation of frontend, backend and database responsibilities provides a foundation for future scalability and additional marketplace functionality.

The next iteration can focus on deeper user testing, improved marketplace discovery, enhanced moderation, notifications, buyer-seller communication and other extensions while retaining the core marketplace workflow.